

A Resident-Streaming Roofline Model for Compute-in-Memory

Abstract—From CPUs to GPUs, the Roofline model has become a textbook tool for revealing both a platform’s performance ceiling and how well a workload matches it. Compute-in-memory (CIM), however, falls outside this picture: fusing computation into the memory array dissolves the compute-memory split underlying the model. We observe a symmetry: conventional processors pair heterogeneous hardware with homogeneous data, whereas CIM pairs homogeneous hardware with heterogeneous data—operands that reside in the array and operands that stream through it. Guided by this symmetry, we derive a resident-streaming Roofline model in which the streaming rate, the resident-write rate, and resident intensity play the roles of peak compute, memory bandwidth, and arithmetic intensity. We define and estimate both rates for ten literature-grounded CIM configurations, from SRAM to flash and emerging non-volatile memories, and count the exact demands of matrix stages in six large language models (LLMs). The critical reuse at which resident writes stop limiting throughput spans more than four orders of magnitude across these configurations, and LLM stages vary as widely in resident intensity. CIM hardware must therefore be matched to its workload, and the model offers quantitative guidance for co-designing both.

Index Terms—Compute-in-memory, roofline model, resident intensity, data reuse, large language models.

I. Introduction

Outline. Use the classical Roofline [?] and Fig. ?? to motivate separate accounting for establishing resident state and evaluating streaming inputs in compute-in-memory (CIM). Position the contribution relative to Jia et al.’s IMC write/compute analysis [?]: their energy-efficiency view already exposes amortization, while this paper develops role- and window-based throughput accounting. Lead from the bound to the native capability map and reuse thresholds in Figs. ?? and ??, with inference matrix stages providing concrete demand examples.

BODY TEXT RESERVED

Introduction: motivation, nearest prior work, and contribution.
Writing round 1: prose not yet drafted.

CONCEPT FIGURE PENDING

- (a) Separate storage and compute
- (b) Resident state and streaming evaluation

Show state establishment/update and input evaluation as distinct services; keep their boundaries visible.

Fig. 1. Operand roles and service boundaries. The planned comparison distinguishes establishing or updating resident state from streaming inputs through that state; weights and historical KV can both be resident.

II. A Resident-Streaming Roofline

A. Operand Roles and Counting Boundaries

Outline. Define an analysis window with logical streaming-input demand Q_S and resident-write demand Q_R , in bytes, at the boundaries sketched in Fig. ??. Count initial loading only when it occurs inside the window, include every update or reload, and match ρ and τ to the same configuration, precision, and service boundaries. Explain that resident state may be weights or historical keys and values; physical encoding, conversion, and required maintenance belong in service accounting.

$$RI = \frac{Q_S}{Q_R}, \quad P = \frac{Q_S}{T} \text{ [Byte/s]}, \quad (1)$$

where T is the window completion time; for $Q_S > 0$ and $Q_R = 0$, define $RI = \infty$.

BODY TEXT RESERVED

Counting boundaries: one compact definition paragraph and a window example.

Writing round 1: prose not yet drafted.

B. Throughput Bound and Resident Intensity

Outline. Derive the two necessary service times and use Fig. ?? to distinguish the meaning of the axes from the familiar Roofline geometry. Identify resident-bound and streaming-bound regions by the tighter term, while treating the curve as an upper bound whose attainability depends on shared resources and scheduling. Use the two compact accounting examples below to connect input throughput to useful operations and expose the effect of changing the residency window.

$$T \geq \max\left(\frac{Q_S}{\rho}, \frac{Q_R}{\tau}\right), \quad (2)$$

$$P \leq \min(\rho, \tau \text{RI}), \quad \text{RI}^* = \frac{\rho}{\tau}. \quad (3)$$

Here ρ and τ are streaming and resident service capacities in Byte/s; $\text{RI} < \text{RI}^*$ is resident-bound and $\text{RI} > \text{RI}^*$ is streaming-bound.

a) Example to develop: useful-operation conversion: One logical 128×128 INT8 matrix-vector multiplication takes 128 input bytes and performs 16,384 MACs. With two useful operations per MAC, the same execution satisfies

$$\frac{32,768 \text{ OP}}{128 \text{ Byte}} = 256 \text{ OP/Byte}, \quad P_{\text{OP}} = 256P. \quad (4)$$

b) Example to develop: three residency windows: For the same matrix and U useful input vectors, $Q_S = 128U$ bytes; only the in-window loading policy changes:

Policy	Q_R [Byte]	RI
Already resident	0	∞
Load once, reuse	16,384	$U/128$
Reload before each use	$16,384U$	$1/128$.

(5)

BODY TEXT RESERVED

Derivation and the two short examples above.
Writing round 1: prose not yet drafted.

CONCEPT FIGURE PENDING

(a) Classical: OP/Byte versus OP/s
memory-bound / compute-bound

(b) Resident-streaming: Byte/Byte versus Byte/s
resident-bound / streaming-bound

Fig. 2. Planned coordinate comparison with the classical Roofline [?]. The two-segment geometry is retained, while demands, capacities, and the meaning of the ridge follow operand roles.

C. From Resident Intensity to Reuse

Outline. Define U as all input vectors served after one complete load, including the first use and allowing accumulation across requests. For $W[N, K]$, use the payload relations below to translate the ridge into the critical reuse shown in Fig. ??, preserving each native matrix and its service boundary. Distinguish unbounded reuse of a finite initial load from a finite, already-resident window with no writes.

$$Q_S = UKb_S, \quad Q_R = NKb_R, \quad (6)$$

$$\text{RI} = \frac{Ub_S}{Nb_R}, \quad \rho = \frac{Kb_S}{\Delta_S}, \quad (7)$$

$$\tau = \frac{NKb_R}{T_R}, \quad U^* = \frac{T_R}{\Delta_S} = \frac{Nb_R}{b_S} \text{RI}^*. \quad (8)$$

The precisions b_S, b_R are bytes per element; Δ_S is a complete-vector service interval and T_R a full-matrix load service time. At equal byte widths, $\text{RI} = U/N$ and $U^* = N\text{RI}^*$.

BODY TEXT RESERVED

Reuse interpretation and the transition to native reference services.

Writing round 1: prose not yet drafted.

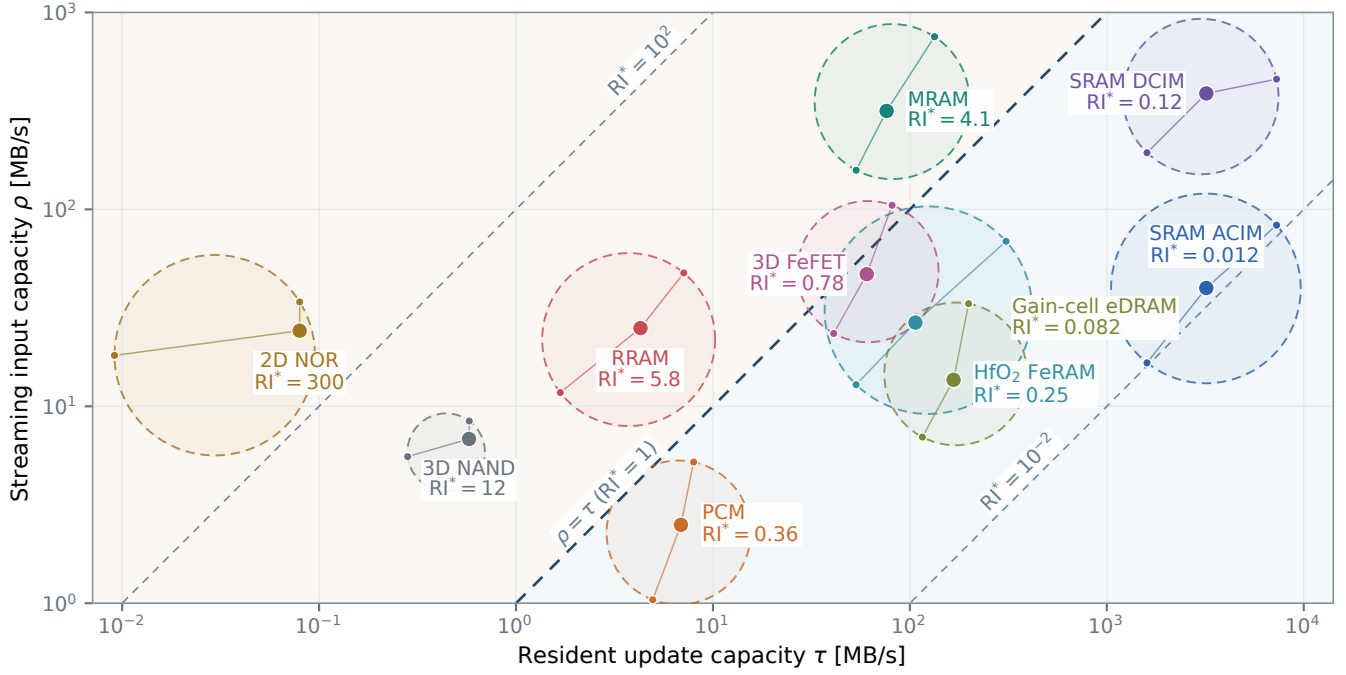


Fig. 3. Streaming capacity ρ and resident-update capacity τ for ten native reference configurations; 1 MB/s = 10^6 Byte/s. Large points are typical scenarios; small points preserve the original paired alternatives. Circles summarize only those finite scenarios, and $RI^* = 1$ is a capacity-ratio reference, not a universal workload boundary. Native sizes and resources differ.

TABLE I
Representative resident intensities at matrix-stage entry

Matrix stage	Qwen3.5 2B	Ministral 3 8B	Qwen3.6 35B-A3B	Hy3	Ling-1T	MiMo V2.5-Pro
QKV, $U = 1K$	0.200	0.167	0.111	0.100	0.100	0.0377
FFN/expert, $U = 1K$	0.222	0.107	0.833	0.306	0.208	0.222
Attention prefill	6.0 / 258.0	10.0 / 514.0	12.0 / 516.0	20.0 / 1028.0	20.0 / 1028.0	35.2 / 1648.0
Attention decode	10.0 / 514.0	18.0 / 1026.0	20.0 / 1028.0	36.0 / 2052.0	36.0 / 2052.0	60.8 / 3286.4

Attention entries are $L = 1K / 64K$; $1K = 1024$. All roles use one byte per element. QKV includes any extra gate; FFN is one dense block or one routed expert. Ling-1T at 64K uses the fixed extended-context configuration.

III. Hardware and Workload Characterization

A. Memory-Based Reference Configurations

Outline. Introduce the ten native analog/digital reference configurations in Fig. ??, grounded in device and macro sources [?], [?], [?], [?], [?], [?], [?], [?], [?], [?] and shared peripheral assumptions [?], [?], [?]. Explain the three paired engineering scenarios and complete service accounting, retaining native matrix dimensions, resources, and update organizations. Read absolute capacity and capacity ratio separately; these are literature-grounded estimates rather than equal-area chip comparisons.

BODY TEXT RESERVED

Native configurations and complete service accounting.

Writing round 1: prose not yet drafted.

B. Inference Matrix Workloads

Outline. Use the six fixed model configurations [?], [?], [?], [?], [?], [?] as structural examples for Table ??, counting common inputs once per matrix stage at one byte per element. Explain QKV intensity U/N_{proj} and FFN intensity $U(D+F)/(3DF)$ using output width N_{proj} (including any gate), hidden width D , and dense-FFN or single-expert width F . For grouped-query attention, use the native head sharing and possibly unequal query/key and value widths: prefill creates each token's KV once, whereas decode appends one token and treats historical KV as resident.

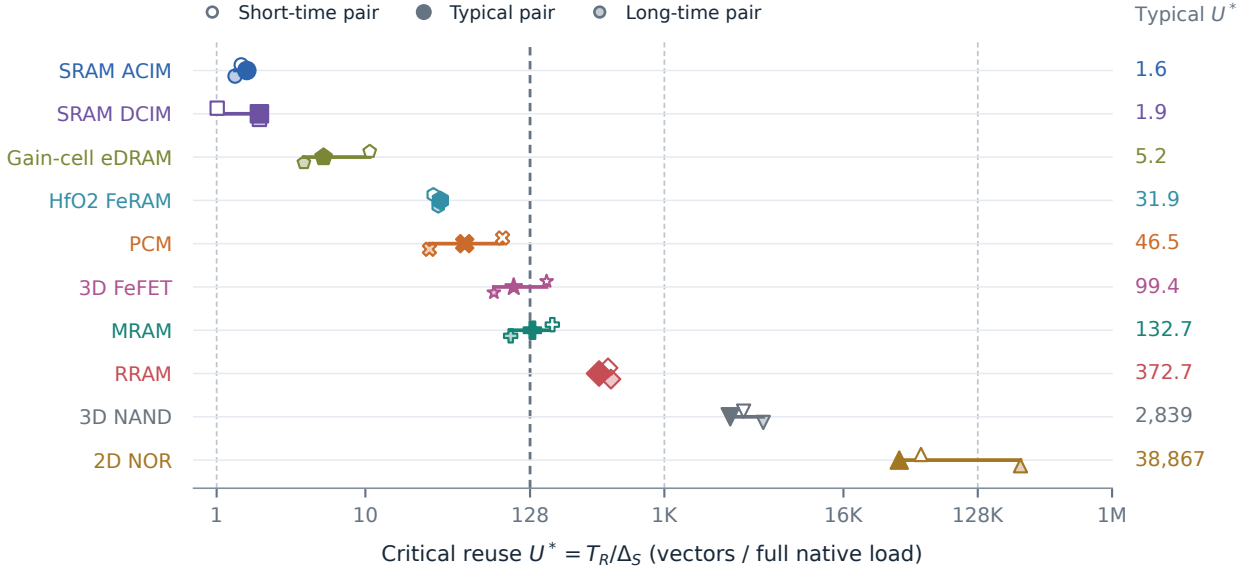


Fig. 4. Critical reuse for one complete native matrix load, $U^* = T_R / \Delta_S = NRI^*$ at equal byte widths. Segments connect three actual paired scenarios, not a continuous feasible range. Dashed lines mark $U = 1, 128, 1K, 128K$ across native shapes; they do not establish a common-shape deployment. $U < U^*$ is resident-bound; $U > U^*$ is streaming-bound. $1K = 1024$.

IV. Reuse Requirements and Design Implications

A. Critical Reuse Across Configurations

Outline. Use Fig. ?? to show that typical full-load thresholds span about 1.6 to 38,867 input vectors, then develop one concrete crossing: at $U = 128$, MRAM is resident-bound in the short and typical scenarios (about 181 and 133), but streaming-bound in the long scenario (about 95). Connect the native threshold to Qwen3.5-2B and MiMo-V2.5-Pro QKV stages using the existing sequential full-tile service reference, for which both sides accumulate the same tile count. Keep this mapping as a component-service bound, with output reduction and other system costs outside its scope.

For native 128×128 tiles, let $m_K = D/128$, $m_N = N_{\text{proj}}/128$, and $M = m_K m_N$:

$$T_{S,\text{comp}} = MU\Delta_S, \quad T_{R,\text{comp}} = MT_R, \quad (9)$$

$$\rho_{\text{eff}} = \rho/m_N, \quad \tau_{\text{eff}} = \tau, \quad (10)$$

$$RI_{\text{eff}}^* = \frac{RI^*}{m_N} = \frac{U^*}{N_{\text{proj}}}. \quad (11)$$

The factor M cancels at service balance. This reference uses one engine, full tiles without padding, and equal U per tile; input replay is included in $T_{S,\text{comp}}$.

BODY TEXT RESERVED

Explain the threshold span and one scenario crossing.

Writing round 1: prose not yet drafted.

B. Design Implications

Outline. Turn a target reuse count into an allowable full-load service time using (??), making Fig. ?? a design requirement rather than a ranking. Revisit Fig. ?? to explain why a lower threshold alone does not imply higher performance: streaming capacity still sets the upper plateau. Contrast weight residency with growing attention state in Table ??; KV append requires its own update-service boundary before applying any full-load threshold.

$$T_R \leq U_{\text{target}} \Delta_S \iff U_{\text{target}} \geq U^*. \quad (12)$$

This is a necessary service-balance condition for reaching the streaming ceiling, not a guarantee of realized throughput.

BODY TEXT RESERVED

Turn a target reuse count into a resident-service requirement.

Writing round 1: prose not yet drafted.

V. Conclusion

Outline. Close with the role-based connection between in-window resident updates and streaming evaluation established by (??). Use Fig. ?? to emphasize the practical outcome: hardware service asymmetry becomes an explicit reuse requirement for residency and update decisions.

BODY TEXT RESERVED

Final conclusion: one short paragraph.

Writing round 1: prose not yet drafted.